

Authentication vs Authorization in Spring Security

Kalariya Heli
Goswami Urvashi
Mevcha Hiya



Objective

Understand the conceptual difference between authentication and authorization in web security.

01

Add Spring Security

Add Spring Security to a Spring Boot project as a dependency

02

Observe Behavior

Observe secured vs unsecured endpoint behavior in the browser

03

Distinguish Concepts

Clearly distinguish authentication from authorization in practice

What is Authentication?

Authentication is the process of verifying who a user is.

Proves Identity

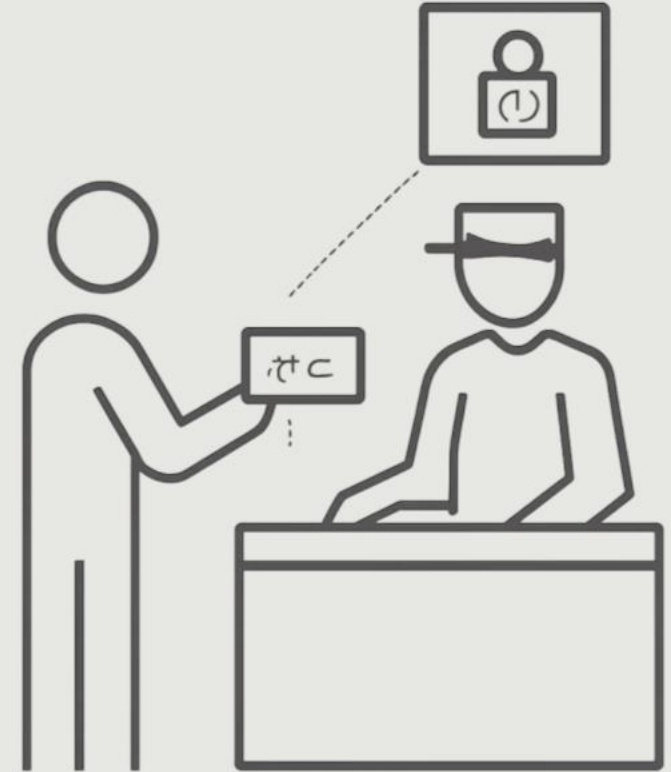
The user must demonstrate they are who they claim to be

Via Credentials

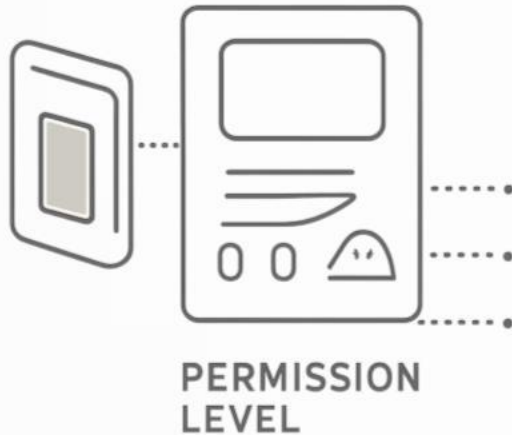
Typically done through a username and password at login

Answers: "Who are you?"

The fundamental identity verification question in any system



What is Authorization?



Authorization is the process of determining what a verified user is allowed to do.

Grants or Denies

Permissions are
assigned or revoked
per user role

Post- Authentication

Only evaluated after
identity has been
confirmed

Answers: "What
can you access?"

Defines the
boundaries of a user's
access

Authentication vs Authorization — Key Differences

Dimension	Authentication	Authorization
Purpose	Verify the user's identity	Control what the user can access
Question Answered	Who are you?	What are you allowed to do?
When It Happens	At login, before access is granted	After authentication succeeds
Example	Entering username and password	Admin can delete users; regular users cannot



A Real-World Analogy



Passport Check = Authentication

Showing your passport at security proves **who you are** — identity verification before entry



Boarding Pass = Authorization

Your boarding pass determines **which gate and seat** you can access — permissions after identity is confirmed

What is Spring Security?

Spring Security is a powerful framework that provides **authentication and authorization** for Java applications.



Auto-Configures on Add

Simply add the dependency — Spring Boot configures security automatically



Secures All Endpoints

Every endpoint is protected by default — no extra configuration needed



Seamless Spring Boot Integration

Works natively with Spring Boot's auto-configuration and starter patterns

Default Behavior in Spring Boot

The moment you add Spring Security as a dependency, these behaviors activate immediately:

1

Auto-Generated Login Page

A default login form is served at `/login`

2

All Endpoints Secured

Every endpoint requires authentication by default

3

Console Password

An auto-generated UUID password prints to the console at startup

4

Default Username

The username is always `user` out of the box

Authentication in Action

What You'll Observe

- Accessing an endpoint without credentials returns **401 Unauthorized** or redirects to login
- Logging in with the console-generated password grants access successfully
- This flow demonstrates **identity verification** in practice

Expected Response

Before login: 401 Unauthorized

After login: 200 OK with resource

- The browser shows a login form — this is Spring Security's default auth page.

Authorization in Action

Expected Response

Authorized: 200 OK

Forbidden: 403 Forbidden

- ☐ Same authenticated user — different outcomes based on assigned roles.

What You'll Observe

- After login, access is granted or denied based on **roles and permissions**
- Responses differ — 403 Forbidden vs 200 OK depending on the user's role
- This demonstrates **permission checking** after identity is confirmed

A faint, stylized illustration of several hands reaching out and interlocking in a supportive grip, rendered in light gray and beige tones. The hands are positioned around the central text, creating a sense of unity and care.

Thank You!